

Computer Science II

Ordinary Differential Equations

Marco S. Nobile, Ph.D.

Bachelor's Degree in Engineering Physics

Ca' Foscari University of Venice

A.Y. 2025-2026



Università
Ca' Foscari
Venezia

Outline

- ODEs
- Explicit methods
 - Euler's method
 - Runge-Kutta family
- Adaptive methods
 - Runge-Kutta-Fehlberg / DOPRI
- Implicit methods
 - Backward Euler
 - LSODA

Ordinary Differential Equations [1/2]

- **Differential Equations (DE)** are a **mathematical framework** to describe and analyse the **dynamics of a system**
- They arise in practically any **natural or social complex system**
- A DE describes the "**rate of change**" of **some quantity** / variable / item in the system
 - E.g., the concentration of a chemical species
 - Note: a DE describes the *change*, it does not provide a closed formula to directly calculate the quantity itself
- The rate of change takes the name of **derivative**

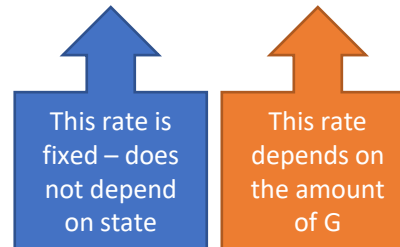
Ordinary Differential Equations [2/2]

- Formally, we write $\frac{d}{dt}x(t) = f(x(t), t)$ where
 - $x(t)$ is the **state** of the system at time t
 - $f(\cdot)$ is any nonlinear function that describes the **change** of the state
 - In ODEs, t is the only independent variable in this definition (usually, it is time)
- Please note that the definition is **general** and the state can be also a vector $\mathbf{x}(t)$, which takes the name of **state vector**
 - In this case, we will have **multiple** equations: **one equation for each variable** of the state vector that must be updated
 - Variables that do not change will have a rate $f(x) = 0$
 - If the variables influence each other (e.g., in a system of reacting molecules) we have a system of **coupled differential equations**

One simple example: gene synthesis

- Consider the constant synthesis of a gene, occurring with rate k_1 , and the degradation of the same gene, occurring with rate k_2
- We now know that we can model this system with two reactions: $\emptyset \xrightarrow{k_1} G, G \xrightarrow{k_2} \emptyset$
- Assuming a well-stirred compartment, we can use the mass-action law and derive the following rate equation for the concentration of species G :

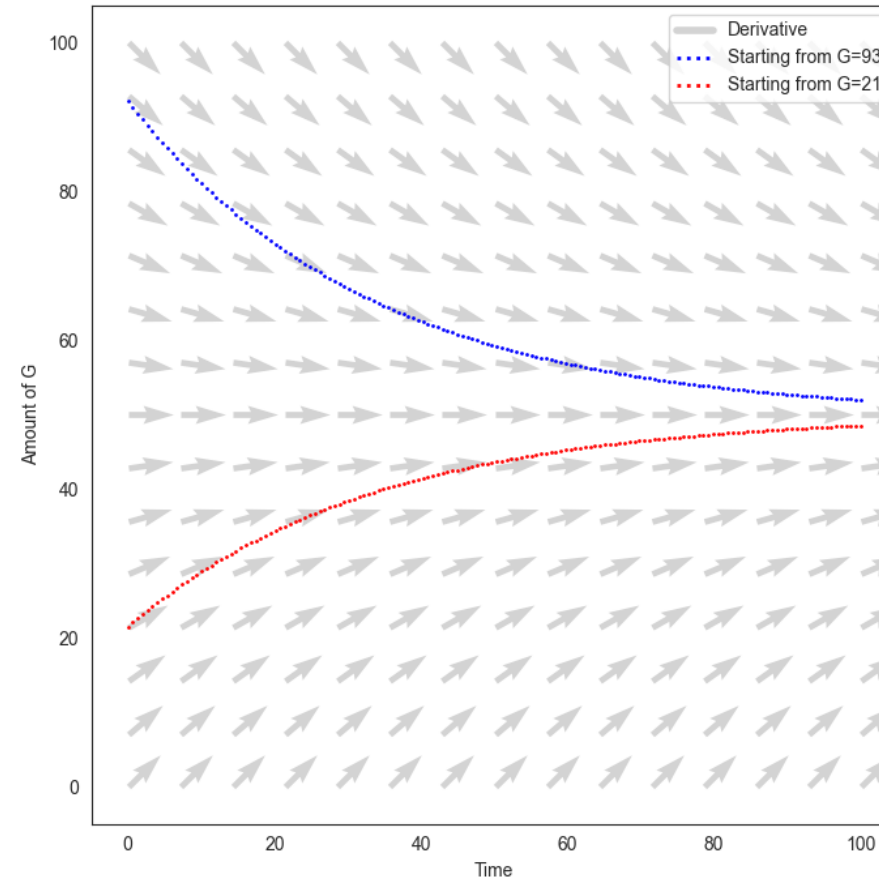
$$\frac{dG}{dt} = +k_1 - k_2 G$$



Let's simulate

- You can find in Moodle a small Python snippet simulating the ODE
- Let's discuss the script and the results together

Plotting derivatives and dynamics



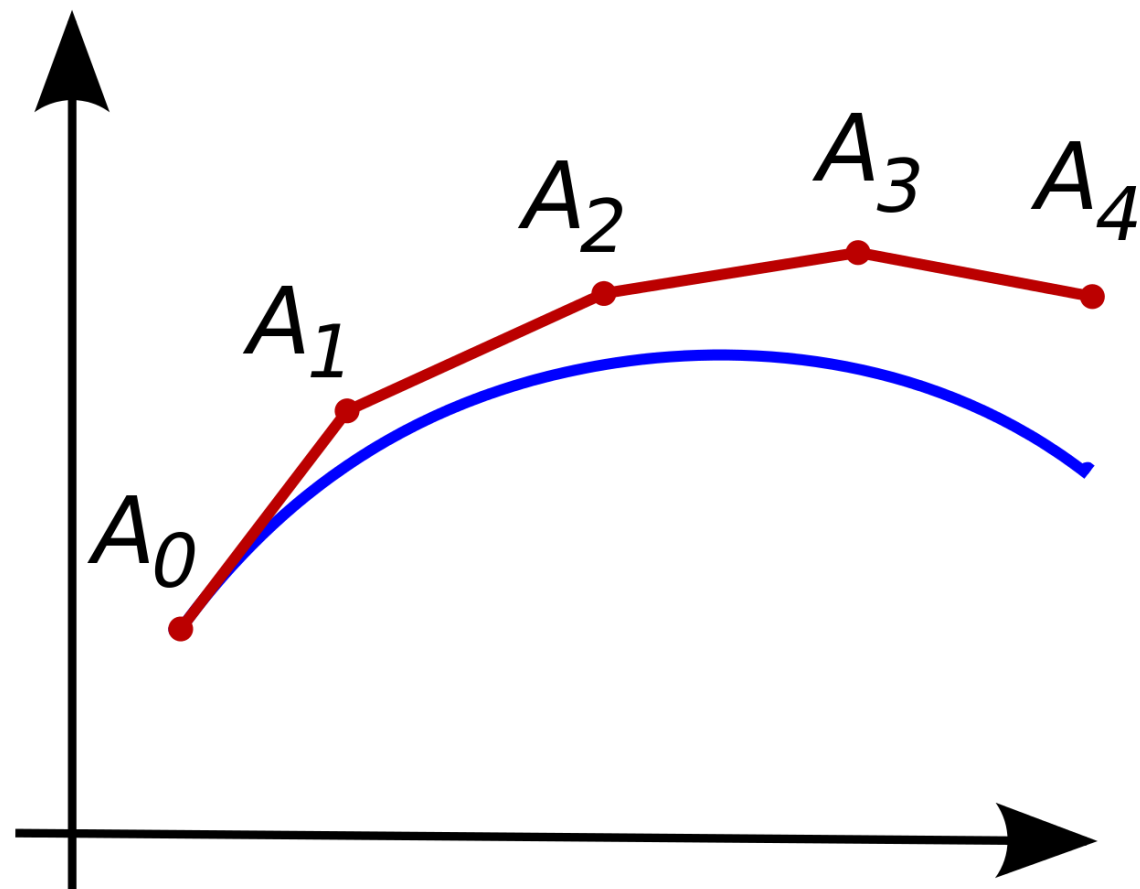
Euler method

- The algorithm I just used is known as **Euler's Method (EM)**
- It is a numerical procedure for **solving ODEs with a given initial value** (i.e., the so-called Cauchy problem)
- EM belongs to the class of **explicit integration methods**
 - That is, it directly calculates the future state of a system from the state of the system at the current time
- The idea is that if $\frac{dx}{dt} = F(x)$ and $\frac{dx}{dt} \approx \frac{x(t+\Delta t) - x(t)}{\Delta t}$ then

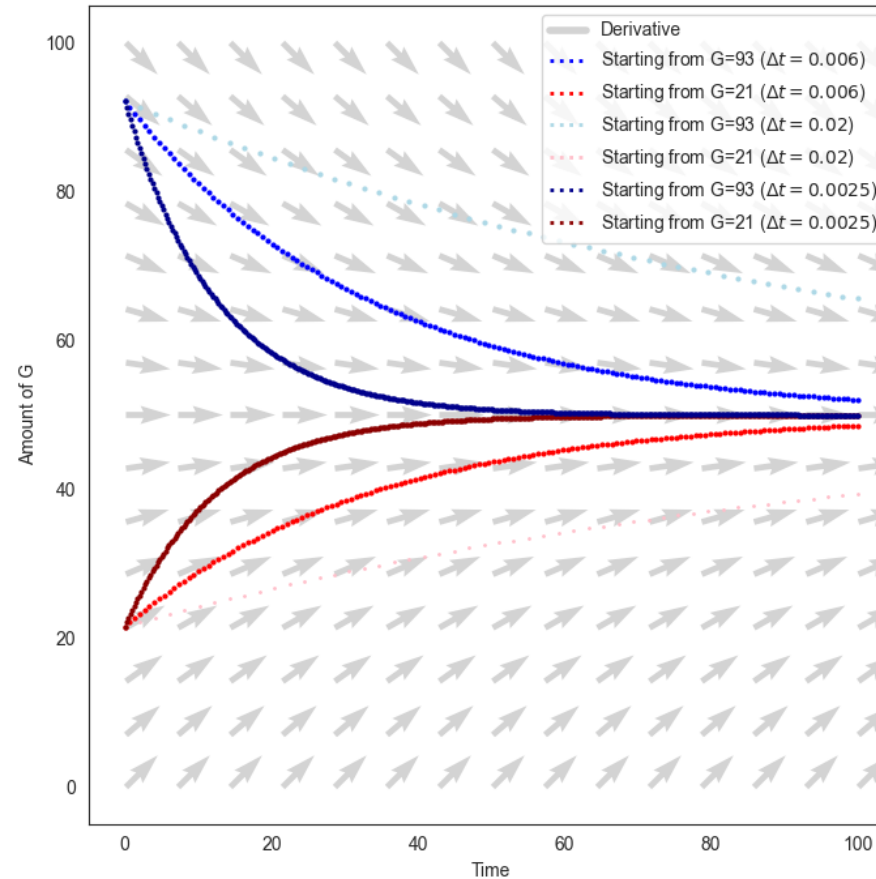
$$x(t + \Delta t) = x(t) + \Delta t \cdot F(x)$$



The derivative is an approximation



Different Δt yield different results

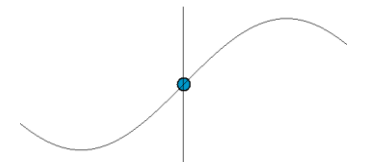


← The smaller Δt ,
the better

However, the
computational effort is
proportional to Δt ...!

Our dearest friend, the harmonic oscillator

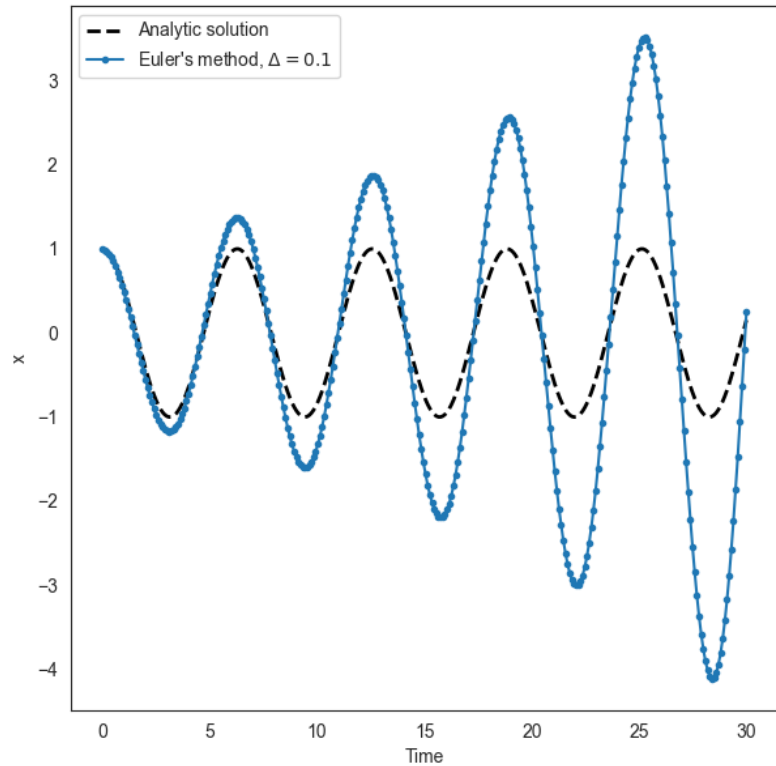
- You have seen it already: it is a relatively simple system with an object attached to a spring, **oscillating** without friction nor damping
 - The system is characterized by **periodic motion** following a **sinusoidal behavior** with constant amplitude A and constant pulse ω
 - That is, $x(t) = A \cdot \cos(\omega t + \varphi)$, where φ is the (constant) phase shift
- We know that the motion is due to the Hook's force: $F = -kx$



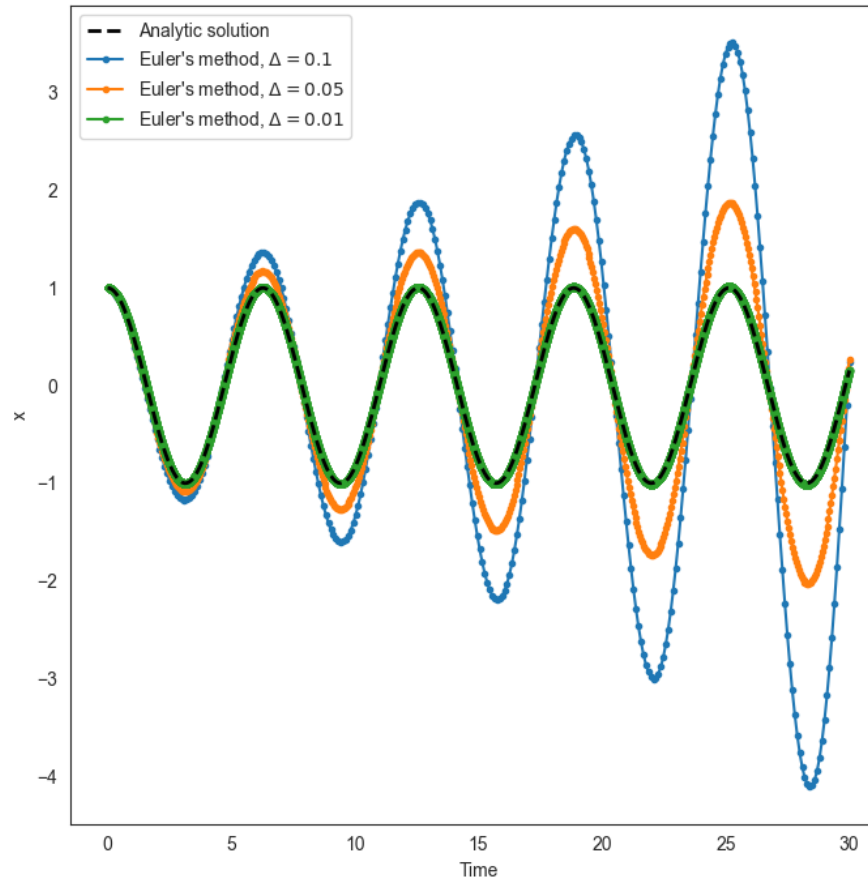
Deriving the ODEs system

- $F = -kx$
- $\omega = \sqrt{\frac{k}{m}} \rightarrow k = m\omega^2$
- $F = ma = m \cdot \frac{d^2x}{dt^2} \rightarrow F = m \cdot \frac{d^2x}{dt^2} = -kx \rightarrow \frac{d^2x}{dt^2} = -\omega^2 x$
- We can introduce a new variable v and split the equation into two ODEs:
 - $\frac{dx}{dt} = v \quad \leftarrow v = dx/dt$
 - $\frac{dv}{dt} = -\omega^2 x \quad \leftarrow a = dv/dt$
- This is the system of ODEs that we are simulating with Euler's method (see code)

Euler's method with $\Delta = 0.1$



Euler's method vs analytic solution

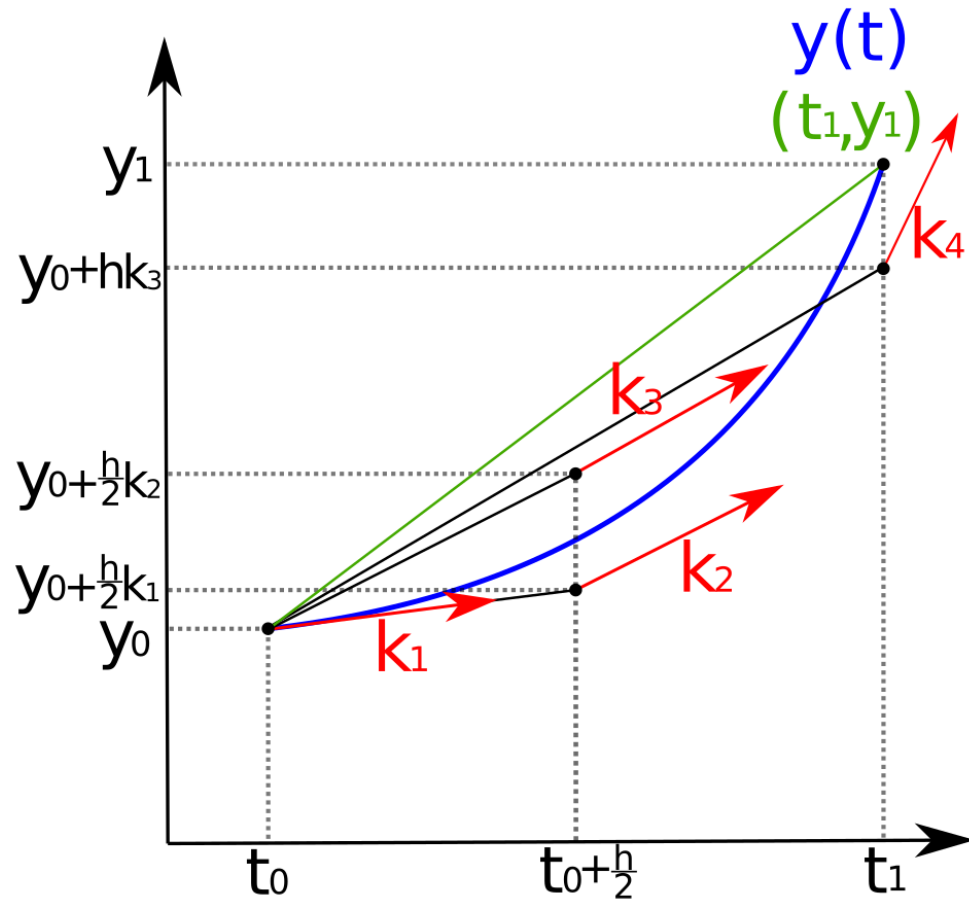


The best choice for Δ is
problem dependent!

A better algorithm: Runge-Kutta

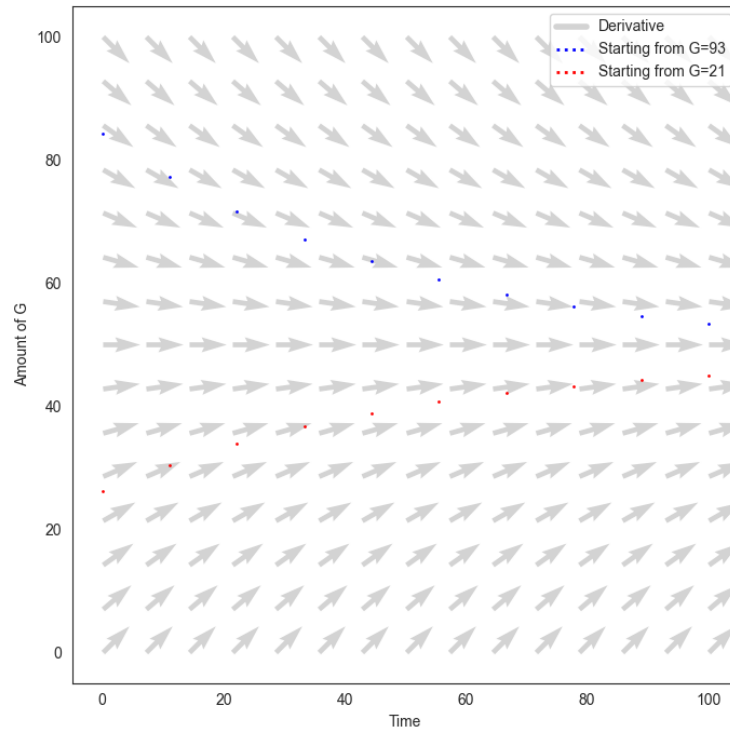
- RK is a **family of methods** to solve the Cauchy problem
 - EM is a special case of RK
 - The most widespread RK algorithm is called "Runge-Kutta 4" (aka **RK4**)
- RK4 approximates the next value by using a **weighted average of four different increments** (hence the name). Given a step size h , they are calculated as follows:
 - $k_1 = f(t_n, y_n)$
 - $k_2 = f\left(t_n + \frac{h}{2}, y_n + h \frac{k_1}{2}\right)$
 - $k_3 = f\left(t_n + \frac{h}{2}, y_n + h \frac{k_2}{2}\right)$
 - $k_4 = f(t_n + h, y_n + hk_3)$
- Finally, $y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)h$

Slopes used by RK4

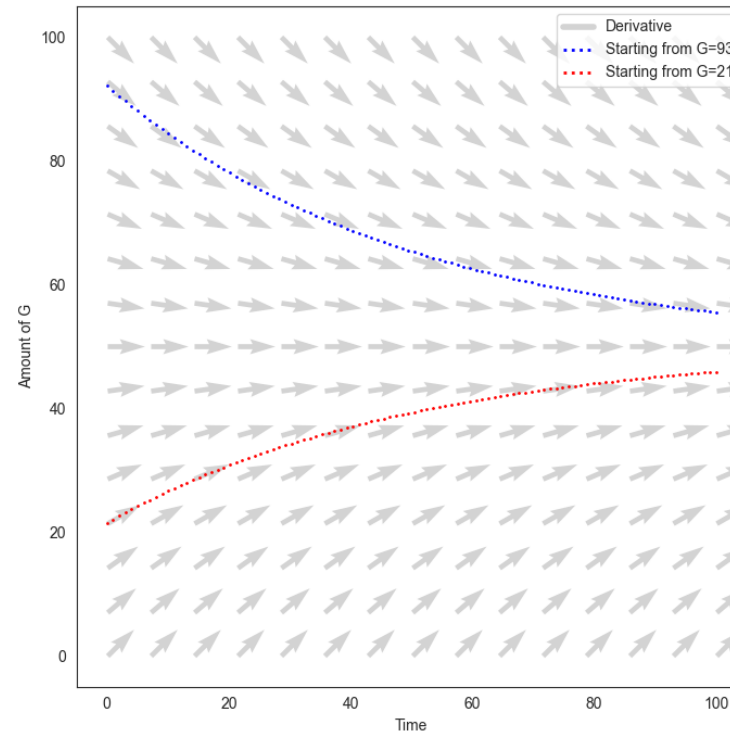


- k_1 is the slope at the beginning, using Euler's method
- k_2 is the slope at the midpoint of the interval, using y and k_1
- k_3 is the slope at the midpoint of the interval, using y and k_2
- k_4 is the slope at the end of the interval, using y and k_3

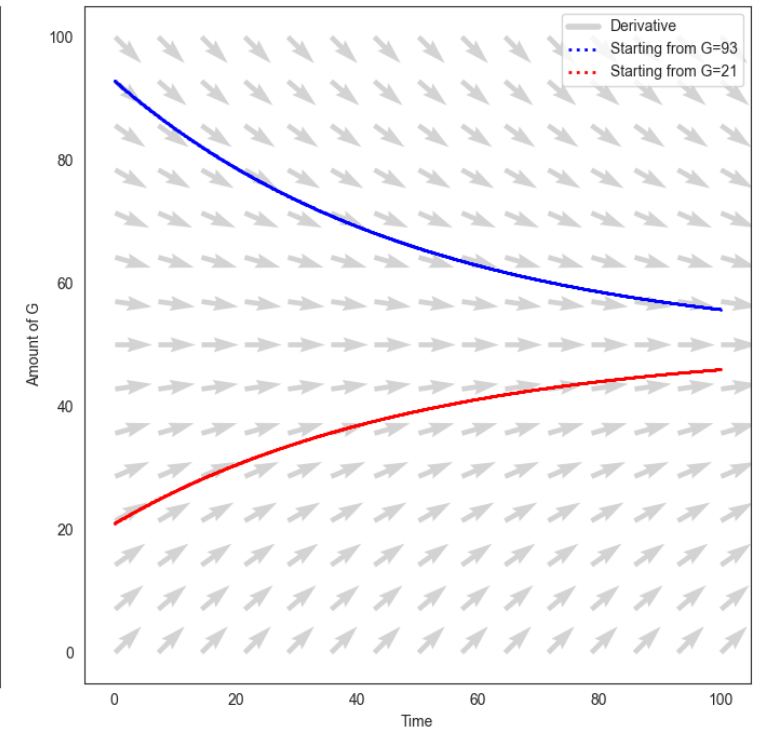
Back to the gene model: Impact of h on RK4



$h=10$



$h=1$



$h=0.1$

Adaptive methods

- Although it is more accurate, the local truncation error¹ of RK4 is still $\mathcal{O}(h^5)$
- It would be useful to **estimate the error of the next step**
 - Then, we are able to systematically **adapt the step size** h so that the error remains below a **user-defined threshold** ε
 - If the error "too high", h is (temporarily?) **decreased** and the step **repeated**
 - If the error is "very low", we can **increase** h to take **longer steps** and improve the performance
- This is the idea of adaptive RK; one example is the so-called **Runge-Kutta-Fehlberg**

¹The norm of the difference between the solution and the approximation after one step

Runge-Kutta-Fehlberg

- RKF (also known as RKF45) avoids the need for step repetition
- Specifically, RKF estimates the **optimal step size** h^* that keeps the **error** below the user-defined threshold ε
- To achieve this goal, RKF (also known as RKF45) performs a step using **two methods**, one with order 4 and one with order 5

RKF45

$$k_1 = hf(t_k, y_k),$$

$$k_2 = hf\left(t_k + \frac{1}{4}h, y_k + \frac{1}{4}k_1\right),$$

$$k_3 = hf\left(t_k + \frac{3}{8}h, y_k + \frac{3}{32}k_1 + \frac{9}{32}k_2\right),$$

$$k_4 = hf\left(t_k + \frac{12}{13}h, y_k + \frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3\right),$$

$$k_5 = hf\left(t_k + h, y_k + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4\right),$$

$$k_6 = hf\left(t_k + \frac{1}{2}h, y_k - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5\right).$$

$$y_{k+1} = y_k + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4101}k_4 - \frac{1}{5}k_5, \quad \leftarrow \text{4-th order RK}$$

$$z_{k+1} = y_k + \frac{16}{135}k_1 + \frac{6656}{12,825}k_3 + \frac{28,561}{56,430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6. \quad \leftarrow \text{5-th order RK}$$

The new optimal step size h^* is finally calculated as: $h^* = h \cdot \left(\frac{\varepsilon \cdot h}{2|z_{k+1} - y_{k+1}|}\right)^{\frac{1}{4}} \approx 0.84 \left(\frac{\varepsilon \cdot h}{|z_{k+1} - y_{k+1}|}\right)^{\frac{1}{4}}$

RKF45

$$k_1 = hf(t_k, y_k),$$

$$k_2 = hf\left(t_k + \frac{1}{4}h, y_k + \frac{1}{4}k_1\right),$$

$$k_3 = hf\left(t_k + \frac{3}{8}h, y_k + \frac{3}{32}k_1 + \frac{9}{32}k_2\right),$$

$$k_4 = hf\left(t_k + \frac{12}{13}h, y_k + \frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3\right),$$

$$k_5 = hf\left(t_k + h, y_k + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4\right),$$

$$k_6 = hf\left(t_k + \frac{1}{2}h, y_k - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5\right).$$

$$y_{k+1} = y_k + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4101}k_4 - \frac{1}{5}k_5, \quad \leftarrow \text{4-th order RK}$$

$$z_{k+1} = y_k + \frac{16}{135}k_1 + \frac{6656}{12,825}k_3 + \frac{28,561}{56,430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6. \quad \leftarrow \text{5-th order RK}$$



This Figure from Wikipedia shows the computational effort during a 3-body simulation with RKF45. Most of the computational effort is spent when the bodies pass close by and are susceptible to numerical error.

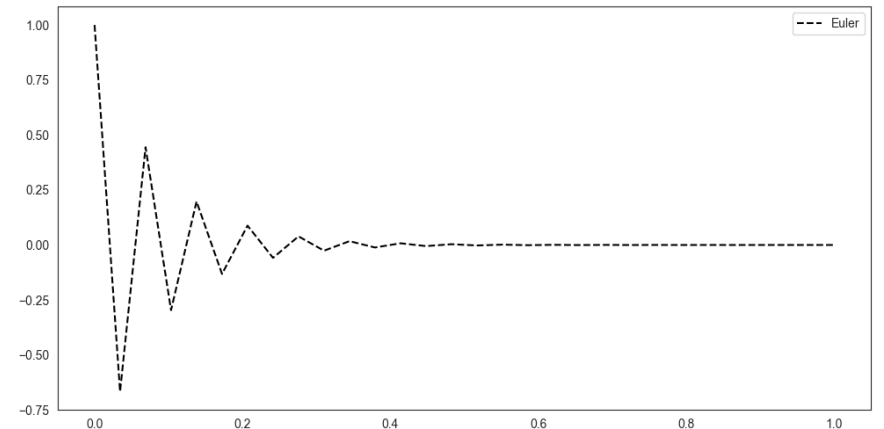
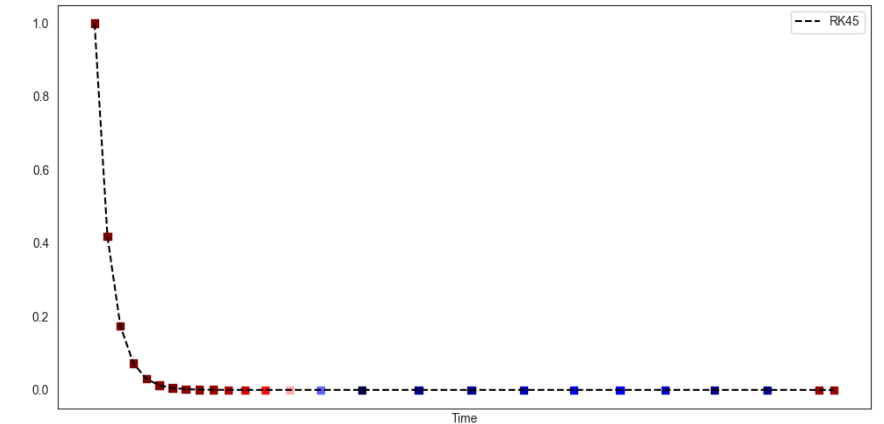
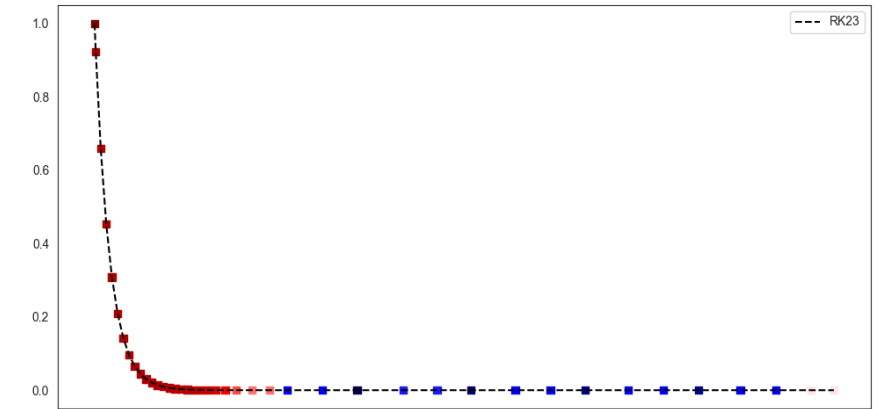
The new optimal step size h^* is finally calculated as:
$$h^* = h \cdot \left(\frac{\varepsilon \cdot h}{2|z_{k+1} - y_{k+1}|} \right)^{\frac{1}{4}} \approx 0.84 \left(\frac{\varepsilon \cdot h}{|z_{k+1} - y_{k+1}|} \right)^{\frac{1}{4}}$$

DOPRI

- The Dormand-Price (aka **DOPRI**) method is a variant of RKF45
 - **More accuracy on the 5-th order method** than RKF45
 - Default method in MATLAB, available in SciPy's ODE integration library (named RK45)
 - See [[Dormand and Price, J Comput Appl Math 1980](#)]
- Example of adaptive RK algorithms: check the python script
- ODE: $y'(t) = -15y(t)$, $y(0) = 1$

Adaptive vs non-adaptive

- RK23 vs RK45 vs Euler
 - RKs, $\varepsilon = 1e-6$
 - Euler, $\Delta = 0.03$
- **RK45 can take longer steps** than RK23, especially during the initial transient
→ faster simulation!
- Euler is **unstable, oscillates**, reaches invalid regions (e.g., <0), behavior is completely different wrt adaptive RK methods



Stiff systems

- Sometimes, in real world systems, some regions of the simulation force the integration method to take **smaller and smaller step sizes**
 - So small that it can even **hit the limits of representability** in a computer
 - Of course, **the smaller the step size, the slower the simulation!**
 - Remember that if take **longer step sizes**, you are **increasing the error** of your simulation
- This kind of systems are called **stiff**
 - There is not a formal definition of stiffness
 - It is more a "phenomenon" than a well-defined property
 - Stick to the intuition and adopt countermeasures...

Implicit methods

- The following modification changes the Euler method and eliminates the stability problems we encountered before:

$$\begin{aligned} y_{n+1} &= y_n + hf(t_n, y_n) \\ &\downarrow \\ y_{n+1} &= y_n + hf(t_{n+1}, y_{n+1}) \end{aligned}$$

- You immediately notice that **we are evaluating the function on the *future* value**, instead of the current value
- This is called "**backward Euler**", and it is an example of **implicit method**

Wait, how?

- $y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}) \rightarrow y_{n+1} - hf(t_{n+1}, y_{n+1}) - y_n = 0$
- We have to solve a non-linear equation
 - Con: **computationally more expensive** than "forward Euler"
 - Pro: **higher stability, longer steps** and **can tackle stiff problems**
- The equation is usually solved using the (possibly, multivariable) **Newton-Raphson** method
 - It is an **iterative** approach
 - Requires the equation to be solved to be **differentiable**
 - Also needs a (reasonable) **starting point**

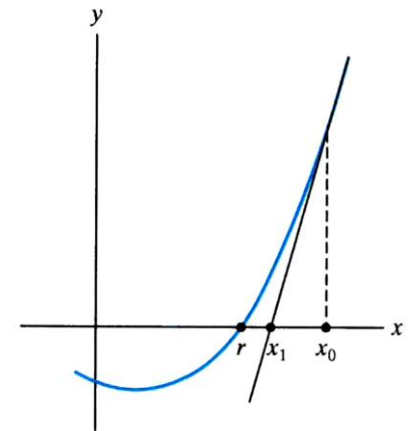


Figure 2.75 The tangent line to $y = f(x)$ at $(x_0, f(x_0))$ crosses the x -axis at $x = x_1$.

Short (?) digression on Newton's method

- Suppose you want to solve the nonlinear equation:

$$\mathbf{F}(\mathbf{Y}) = \mathbf{0}$$

← Note that they
are all vectors

- The following formula is called the Newton iteration:

$$\mathbf{Y}_{n+1} - \mathbf{Y}_n = - \left(\frac{\partial \mathbf{F}}{\partial \mathbf{Y}} (\mathbf{Y}_n) \right)^{-1} (\mathbf{F}(\mathbf{Y}_n))$$

- n here denotes the iterations of the Newton method, not the integration step
- If your initial guess for $\mathbf{Y}_0$ is good, it converges quite rapidly
- The partial derivatives must be interpreted as the **Jacobian** matrix whose components are $J_{ij} = \frac{\partial F_i}{\partial Y_j}$

VODE, LSODA

- VODE and LSODA are **state-of-the-art** initial value ODE solvers for both stiff and nonstiff systems
 - VODE requires the user to **specify whether the system** is stiff or not
 - [Brown *et al.*, J Sci Stat Comput 1989]
 - LSODA **automatically switches between stiff and non-stiff** (i.e., implicit) methods
 - [Petzold, J Sci Stat Comput 1983]
 - The **estimation of stiffness** itself is **time-consuming** (it is detected by calculating the error and the step size → all wasted computation)
 - If you know your system well VODE could be more effective
- Both algorithms use Newton-Raphson for implicit algorithms, which requires **additional calculations** (in particular, to evaluate the Jacobian matrix)
- In order to use VODE and LSODA, you need to **calculate the Jacobian**

Calculating the Jacobian

- SciPy gives access to LSODA: to use it, we must implement **a new function** that **evaluates the Jacobian** on the current values using the systems parameters
 - LSODA can actually internally approximate the Jacobian but it is **computationally expensive**
- There are **two ways** to implement the function for the Jacobian matrix:
 - Calculate the partial derivatives **by hand**
 - Reinvent the wheel (i.e., create your own symbolic mathematics)
 - Let Python do the job: use **SymPy**

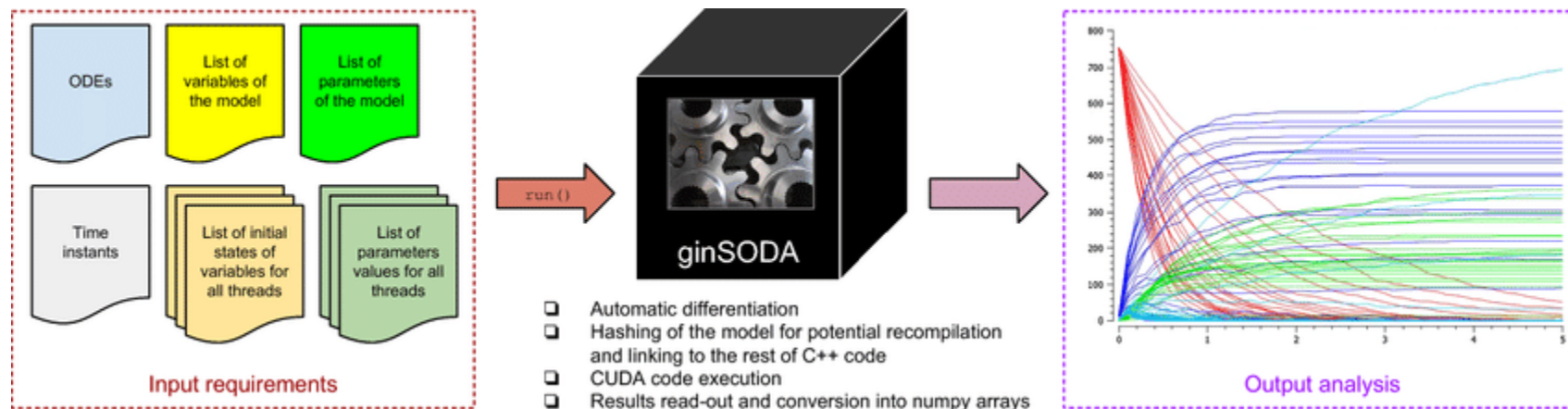
SymPy



- SymPy is a Python library for **symbolic mathematics**
 - It can calculate limits, Taylor series, and of course **differentiation** and integration
- How?
 - `from SymPy import *`
 - Create a `Symbol` object for each variable
 - Create a function for each ODE
 - For each function, use the `.diff(·)` method to get the derivative with respect to `Symbol` •
 - We can exploit the `.lambdify()` method to make the derivatives actual functions
- See example script

ginSODA

- ginSODA: massive parallel integration of stiff ODE systems on GPUs
 - ginSODA takes an arbitrary system of coupled ODEs
 - The ODEs are **differentiated using SymPy** in order to generate the Jacobian matrix
 - The Jacobian is then converted to C code, compiled as GPU code (using CUDA) and linked to a GPU version of LSODA in order to **speedup a massive number of simulations**
 - [Nobile *et al.*, J Supercomp 2018]



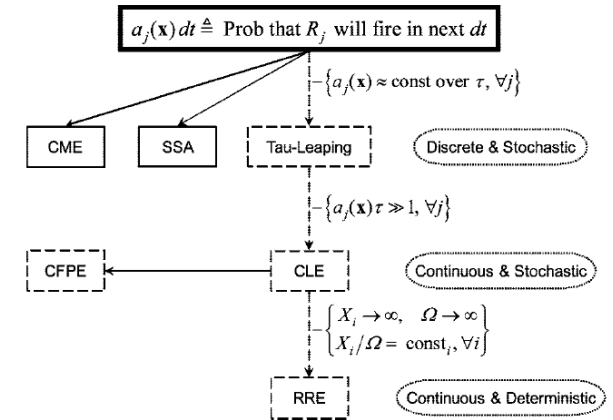
So... It. Can. Be. Done!



More about GPU
simulation at the
end of the course

One final word about stochasticity

- What shall I use? Markov chains or ODEs?
 - Rule of thumb: if you have a few items in your system **noise is probably not negligible** → go stochastic
 - If you are close to the thermodynamic limit use **ODEs** (i.e., reaction rate equations)
 - See [\[Gillespie, J Phys Chem 2009\]](#)
- There exist some "intermediate" regimes that can be simulated in alternative ways
 - i.e., approximate stochastic simulation
 - Stochastic Differential Equations (SDEs)



Summary

- When noise is negligible, we can create use a model with (coupled) ODEs to investigate the dynamic behavior of our system of interest
- Analytic solutions are seldom available, so we must rely on numeric ODE solvers
- We can use small step size and higher-order explicit Runge-Kutta methods to keep the error under control
- We can also use adaptive step size to balance the computational effort where is really necessary
- However, stiffness can cause the step size to become way too small, so that implicit integration is the only feasible approach
- Implicit integration mandates the calculation of the Jacobian matrix
- Python provides access to LSODA, which can automatically switch between explicit and implicit solvers